

Partial Offloading Strategy Based on Deep Reinforcement Learning in the Internet of Vehicles

Shujuan Tian[✉], Xinjie Zhu, Bochao Feng, Zhirun Zheng[✉], Haolin Liu[✉], *Member, IEEE*,
and Zhetao Li[✉], *Member, IEEE*

Abstract—Driven by the increasing demands of vehicular tasks, edge offloading has emerged as a promising paradigm to enhance quality of experience (QoE) in Internet of Vehicles (IoV) networks. This approach enables vehicles to offload computation-intensive tasks to edge servers, resulting in reduced computation delays and lower energy consumption. However, traditional binary offloading limits the efficiency of edge offloading. To address this gap, we propose a partial offloading strategy that jointly optimizes the offloading ratio, computation, and communication resources in IoV. Recognizing the varying priorities of vehicular tasks regarding task delay and energy consumption, we formulate two distinct scenarios: one focused on minimizing delay and the other on minimizing energy consumption. Furthermore, we employ a reinforcement learning approach to establish a multi-dimensional joint optimization function by setting different objectives for each scenario. Based on this framework, we introduce a multi-state iteration deep deterministic policy gradient algorithm (SIDDPG), which effectively determines task partitioning and resource allocation. Simulation results demonstrate that the proposed algorithm outperforms benchmark schemes in terms of task delay and energy consumption.

Index Terms—Internet of Vehicles, partial offloading, reinforcement learning.

I. INTRODUCTION

THE development of the Internet of Vehicles (IoVs) enables a variety of vehicular tasks, such as autonomous driving. These tasks are typically computing-intensive and require substantial resources for execution [1]. However, due to cost and physical size constraints, onboard vehicular systems have limited resources, which may not meet these requirements. Cloud

computing provides a solution by allowing vehicular tasks to access adequate cloud resources in a cost-effective and reliable manner [2]. This approach effectively alleviates the resource constraints faced by vehicular terminals. Nonetheless, cloud computing introduces significant latency when transmitting data to the public cloud over wide-area networks [3].

To address these challenges, Mobile Edge Computing (MEC) has emerged as a highly promising solution that enhances the efficiency of vehicular systems. With MEC servers situated in close proximity to vehicles, edge processing not only minimizes the latency associated with data transmission but also reduces task computation delay. By processing data closer to the source, vehicles can access necessary resources more quickly, enabling real-time decision-making and enhancing the performance of applications such as autonomous driving. This localized approach also alleviates the burden on cloud infrastructure, reducing network congestion and improving overall system reliability [4].

Traditional MEC offloading schemes treat vehicular tasks as holistic units, resulting in a binary choice between local on-board processing and complete offloading to the base station. This all-or-nothing approach restricts collaboration between the end and edge sides. In practice, a vehicular task often consists of multiple components, which opens the door for fine-grained (partial) computational offloading [5]. To enhance processing efficiency, task partitioning can be utilized. This method divides each vehicular task into multiple subtasks, allowing them to be assigned to local devices, edge servers, or cloud servers for concurrent execution. Once the subtasks are completed, their results can be wirelessly transmitted and aggregated within the vehicle. This strategy significantly reduces task latency and energy consumption, particularly in scenarios involving large tasks or datasets [6], [7], [8], [9].

Reinforcement learning (RL) is widely recognized for its effectiveness in multidimensional objective optimization. These algorithms can be employed to develop collaboration strategies that facilitate multi-objective optimization, resulting in enhanced task processing efficiency and a reduced computational burden [10], [11]. In the context of IoV, enhancing existing RL algorithms to accommodate dynamic task segmentation schemes and resource allocation strategies can significantly improve vehicle task processing within the transportation ecosystem and enhance the quality of user services.

In this paper, we aim to design a partial offloading strategy based on deep reinforcement learning in IoV networks. How-

Received 13 January 2024; revised 21 November 2024; accepted 17 February 2025. Date of publication 25 February 2025; date of current version 5 June 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62172349, Grant U24A20247, and Grant 62372396, in part by Major landmark innovation demonstration project in Hunan Province under Grant 2022XK2301, in part by National Natural Science Foundation of Hunan Province under Grant 2023JJ30597 and Grant 2023JJ30595. Recommended for acceptance by H. Shen. (Corresponding author: Haolin Liu.)

Shujuan Tian, Xinjie Zhu, Bochao Feng, and Haolin Liu are with the School of Computer Science and School of Cyberspace Security, Xiangtan University, Xiangtan 411105, China, also with the Hunan International Scientific and Technological Cooperation Base of Intelligent Network, Xiangtan University, Xiangtan 411105, China, and also with the Key Laboratory of Hunan Province for Internet of Things and Information Security, Xiangtan University, Xiangtan 411105, China (e-mail: sjtianwork@xtu.edu.cn; 202121623071@smail.xtu.edu.cn; fbc19981221@glut.edu.cn; liu.haolin@foxmail.com).

Zhirun Zheng is with the School of Mathematics and Computational Science, Xiangtan University, Xiangtan 411105, China (e-mail: zhengzhirun2019@gmail.com).

Zhetao Li is with the College of Information Science and Technology, Jinan University, Guangzhou 510632, China (e-mail: liztchina@hotmail.com).

Digital Object Identifier 10.1109/TMC.2025.3543976

ever, several critical issues must be addressed beforehand. *First*, task partitioning is influenced by multiple factors, including edge resource allocation for computation and communication, as well as optimization objectives related to task delay and energy consumption. Formulating a partial task offloading problem that incorporates these resource allocation decisions and objective preferences is a significant challenge in IoV networks. *Second*, when applying RL approaches to tackle the partial offloading issue, determining task segmentation parameters proves difficult. The parameters are intricately linked to wireless communication conditions and edge computation resources. Thus, finding an equilibrium to establish the edge processing ratio requires careful consideration.

To address the issues mentioned above, the contributions of our work are outlined as follows:

- We propose a novel offloading model based on task partitioning in IoV networks. The objective of this model is to maximize the utilization of computational and storage resources within the system, which includes both vehicles and edge servers. To achieve this, we select different processing methods depending on whether the tasks can be partitioned.
- We formulate a joint optimization problem for task partitioning and resource allocation in MEC systems. This problem is then divided into two interrelated subproblems: the task partitioning subproblem and the resource allocation subproblem. Additionally, we introduce the concept of a task equilibrium point as an indicator to optimize the task partitioning ratio.
- We propose a multi-state iteration deep deterministic policy gradient (SIDDPG) algorithm. The algorithm is based on the DDPG framework and is specifically designed to deal with the sequential correlation between subproblems. SIDDPG splits actions, and through iterative training, gradually optimizes the action strategy of each neural network at different stages. In each round of training, actions a_1 and a_2 correspond to different evaluation networks, and their effects are evaluated respectively. Then, the soft update strategy is used to update the target network parameters to generate a more efficient task division and resource allocation strategy.
- We conduct simulations to validate the effectiveness of the proposed algorithm. Through experimental analysis, SIDDPG algorithm has improved in delay optimization and energy consumption reduction, which provides a new scheme for task offloading in mobile edge computing.

The rest of the paper is organized as follows. Section II summarizes the related works. Section III and Section IV present the system model and problem transformation, respectively. Section V designs deep reinforcement learning-based solutions. Section VI presents the simulation model and performance evaluation. Section VII makes the conclusion. Finally, Section VIII presents the acknowledgement.

II. RELATED WORK

Binary offloading refers to the situation where vehicular tasks have only two choices: either they are computed locally or

offloaded to the MEC server, without any intermediate options. In recent years, numerous studies have emerged with the aim of optimizing the efficiency of task offloading. The majority of existing research focuses on binary task allocation, particularly for tasks involving sensitive private data or those with significant sequential relevance. These studies emphasize externally assisted offloading or algorithmic optimization techniques to minimize processing latency and energy consumption in vehicular tasks.

In [12], the authors introduced an approach to establish a cooperative distributed computing architecture for IoT edge multi-access networks in Vehicular Networking. This architecture leverages vehicles as edge computing resources to guarantee low latency and application-level performance. Y.M.Sapu et al. [13] proposed an approach that utilizes the correlation between vehicle tasks to enhance their computational efficiency. This approach involves developing a network demand prediction mechanism where a content server serves as a centralized node to gather task information from all nodes. In [14], vehicle paths are predicted considering vehicle mobility, and tasks are offloaded across domains using vehicle-vehicle communication to minimize the transmission latency of tasks. In [15], the authors exploit the versatility of UAVs by leveraging them in conjunction with edge servers to collaborate with road test units for task offloading. They employ the MADDPG algorithm to obtain vehicle correlation and make resource allocation decisions. F. Busacca et al. [16] represented a three-tier architecture including vehicle domain, MEC domain, and backhaul network domain is proposed to make allocation decisions to stakeholders for load balancing through reinforcement learning algorithms. However, the aforementioned works primarily focus on reducing communication delays and auxiliary computations, neglecting the potential for fully utilizing the vehicle's own computational resources and the collaborative computational capabilities of the MEC server.

The selection of edge servers is a critical part of the compute offloading process, which not only affects the latency and energy consumption of task processing, but also affects the pricing policy. In many studies related to task offloading, edge server selection is based on geographic location, such as selecting the nearest server by driving direction. However, this selection method ignores the importance of pricing factors in decision-making. In fact, different pricing strategies can lead users to choose a server that costs less but may be slightly farther away, or a server that is relatively less expensive during peak hours to balance cost and performance. As a result, considering pricing decisions in an offload strategy is an area worth digging into. In [17], the authors deeply explore the impact of pricing MEC policies on offloading decisions, and shows that the pricing strategy of edge server resources can significantly affect users' offloading decisions, which in turn affects the performance and resource utilization efficiency of the entire system. In [18], the authors introduce an AGORA framework for multi-provider multi-user multi-access edge computing, formulate the joint optimization of data offloading, computing resource allocation and pricing as a multi-leader multi-follower Stackelberg game. In [19], the authors investigate minimizing users' maximum delay in an RSMA-based multi-server MEC system, where

a non-convex problem is formulated and approach with task assignment, radio and computing resource allocation is derived. On top of that, we can further extend the model to account for the impact of the edge server's pricing policy on the offload decision. By simulating the offloading behavior under different pricing strategies, we can more comprehensively evaluate the effectiveness and adaptability of the proposed model in practical applications. This not only helps to improve the quality of service experience for users, but also provides a theoretical basis for the reasonable allocation of edge computing resources and the formulation of pricing strategies.

Building upon the optimization objectives of the aforementioned studies, we propose an extension to the concept of task partitioning. We introduce an online fine-grained partitioning technique that enables the full utilization of computational power from both the local device and the MEC server for each individual task. Based on the MAUI platform presented in [20], a system that offloads fine-grained mobile code to the infrastructure is designed. The system utilizes a code hosting platform to support fine-grained task partitioning. Unlike cellular networks, wireless technologies in the in-vehicle environment generally utilize the contention-based IEEE 802.11p standard, which makes partial offloading more reasonable [21], [22]. Partial offloading has also gained the attention of some researchers due to a range of communication technologies as well as task segmentation platforms.

In terms of algorithm optimization, Z. Kuang et al. [23] proposed an iterative algorithm for the POSP joint problem for the joint problem of partial offloading scheduling and resource allocation for a MEC system with multiple independent tasks, guaranteeing that the weighted sum of the execution latency and energy consumption is minimized subject to the task transfer power constraints. The correlation between the front and back partial subtasks is considered in [24], and a partial program offloading (PPO) scheme is proposed to minimize the overall time consumption of the multi-edge server system to complete all user offloading tasks. In [25], X. Deng, J. Yin considers et al. proposed a deep reinforcement learning framework for supporting delay-aware partial computation task offloading in Industry 5.0, which regularizes the task division sub-blocks to a specific ratio, and optimizes the overall system efficiency by means of the DDPG algorithm.

As for optimization goals, the main focus is on how to reduce the total latency of tasks as well as reduce the energy consumption of vehicles and edge servers. The article [26] performs a joint optimization of the task partition ratio, the device transmit power and the device computational speed to minimize the energy consumption of the device and the task execution delay. On the other hand, S. E. Mahmood et al. [27] investigated the resource allocation problem for multi-user MEC offloading systems based on time division multiple access and orthogonal frequency division multiple access, and formulated the optimal resource allocation problem as a convex optimization problem minimizing a weighted sum of mobile energy consumption under computational delay constraints. Based on the derived offloading priority function, priorities are generated for users

based on their channel gains and local computational energy consumption. In article [28], authors define a net utility, based on the architectural diagram of the task components, determine which components need to be offloaded and the priority of offloading them, weighing the energy saved by the mobile device against the constraints of communication latency, overall application execution time, and component prioritization. The problem of task partitioning and user association in MEC systems, aiming at minimizing the average latency of all users, was proposed in [8], and a dual decomposition-based approach was proposed to solve it. Article [29] considers resource competition among mobile users on the basis of the single-user computational partial offloading problem, formulates the multi-user computational offloading problem as an np-hard mixed-integer linear programming problem, and designs an iterative heuristic MEC resource allocation algorithm to dynamically make offloading decisions. And in the article [30], a three-tier task partitioning is proposed by combining the edge-cloud model architecture as well as the task partitioning structure, where the local, the edge server as well as the cloud perform the task offloading computation to minimize the latency. M. Qin et al. [31] also considered multi-part division during transmission, by performing task division after task uploading through multiple base stations and then integrating them at the edge server side, so as to reduce the latency during wireless transmission.

In addition to the vertically cooperative partial offloading mentioned above, the horizontal joint offloading between vehicles has also made some progress. The article [32] makes use of the arithmetic aggregation area formed by car-market vehicles in traffic congested road sections to carry out joint vehicle-vehicle offloading, where the tasks are divided and then assigned to different idle vehicles for computational processing, and particle swarm optimization algorithms (PSOs) are used to solve them.

Building upon the existing research on binary offloading, recent advancements in autonomous driving technology and fine-grained code partitioning platforms have made partial offloading increasingly feasible. This advancement has the potential to enhance processing efficiency within MEC systems. In this paper, we propose a multi-state iteration offloading scheme based on the Deep Deterministic Policy Gradient (DDPG) algorithm. The objective of this scheme is to optimize resource allocation strategies within the partial offloading model by effectively combining on-board resources with the resources offered by edge servers. The ultimate goal is to achieve optimal average vehicle latency.

III. SYSTEM MODEL

In our envisioned scenario, a bi-directional highway lane is equipped with i evenly distributed MEC servers (denoted as $M = \{m_1, m_2, \dots, m_i\}$) located in the middle. These MEC servers serve as powerful wireless edge offloading points for self-driving vehicles. After the computation is completed, the MEC servers wirelessly transmit the results back to the vehicles. We denote the computational, storage, and spectral resources provided in the node m by $[C_m, S_m, F_m]$, respectively. Within the coverage area of the MEC servers, let $\{n_j | j = 1, \dots, n\}$ to

TABLE I
SUMMARY OF THE KEY NOTATIONS

Parameter	Sign	Parameter	Sign
i -th MEC	m_i	j -th Task	n_j
CPU cycles of task n	c_n	Data of task n	s_n
Max latency of task n	d_n	Local compute resource	c_n^{UE}
MEC compute resource	C_m	MEC storage resource	S_m
MEC spectral resource	F_m	Offloading ratio	a_n
Spectrum ratio	β_n^m	Compute ratio	χ_n^m
Spectral efficiency	h_n^m	Vehicle compute time	$T_{n,com}^{loc}$
MEC compute time	$T_{n,com}^{mec}$	Transmission time	T_n^{tran}
Offloading time	T_n^{mec}	Transmission rate	V_n^m
Total delay	T	Transmission power	p_n^{tran}
Vehicle compute power	$P_{vehicle,n}^{com}$	MEC compute power	$P_{edge,m}^{com}$
Vehicle power	$E_{n,com}^{loc}$	Transmission energy	$E_{n,tran}$
MEC compute energy	$E_{n,com}^{mec}$	Total energy	E_{sum}

represent the j -th self-driving vehicle traveling at the unused speed within the highway. These vehicles can moving in the same or opposite direction on each side of the server. They generate different tasks $\{task_j | j = 1, \dots, n\}$ with different sizes and overheads during the movement of the vehicles. These tasks can be processed either locally on the onboard server or offloaded via wireless transmission to the MEC server for processing. We list the key notations as Table I for better readability.

A. Task Model

In the service area of the MEC server, we assume that each vehicle randomly generates computational tasks that need to be processed [33]. Each vehicle user has only one independent task that needs to be computed within a specific time period. Therefore, we describe the task t_n generated by the n th vehicle as a tuple $[c_n, s_n, d_n]$, where c_n denotes the necessary CPU cycles for task completion, s_n corresponds to the task's associated data size, and d_n signifies the maximum permitted latency for task execution.

The images obtained by the vehicle can be divided into different layers, and video stream tasks can be divided into two groups of images. These layers and images can be processed by GPUs locally and on the MEC, after extracting the required feature values, and feedback to the vehicle. So we introduce the task division ratio α_n to enhance the synergy between the resources of vehicle and MEC. Finally, the vehicle concatenates the individual feature values locally. In summary, the initial task $task_n$ is divided into two subtasks based on the ratio α_n , denote the offloaded and local computation parts by $\alpha_n task_n^{mec}$ and $task_n^{loc}$, respectively:

$$task_n^{mec} = [\alpha_n c_n, \alpha_n s_n, d_n], \quad (1)$$

$$task_n^{loc} = [(1 - \alpha_n) c_n, (1 - \alpha_n) s_n, d_n], \quad (2)$$

where $\alpha_n \in [0, 1]$ represents the percentage of vehicle tasks that are chosen to be offloaded to edge servers, the task is calculated entirely locally when α_n is equal to 1, and it is all offloaded

when it is equal to 0, intermediate values representing partial offloading.

After the completion of the different subtasks, their results are transmitted to the vehicle and combination. The time required for aggregation is dependent on the task's attributes and size. In this paper, the task has only one split-combination operation. Therefore, the latency of this operation is already included in the vehicle's maximum tolerable latency, it does not need to be considered in subsequent calculations.

B. Communications Model

There is a partial overlap in the server ranges of neighboring MEC base stations on both sides of the roadway, which ensures that the wireless transmission signal can effectively cover the entire road. The transmitting and receiving frequencies of the base stations are fixed. The uplink and downlink transmission channels between the vehicle and the base station are symmetric and influenced by the spatial distance between them and stationary noise. Let p_n^{tran} represent the transmit power of vehicle n . The spectral efficiency of the associated vehicle n under the coverage of the mounted base station is calculated as:

$$h_n^m = W \log_2 \left(1 + \frac{p_n^{tran} G_n^m}{\sigma^2} \right), \quad (3)$$

where W is the channel bandwidth, G_n^m is the channel gain between the vehicle and the base station, the value of which varies continuously with the distance between the vehicle and the base station, and σ^2 is the power spectral density of Gaussian White Noise (AWGN) under the road section. Let $\beta_n^m \in [0, 1]$ be the proportion of spectrum resources allocated to vehicle n by base station m . Therefore, the data rate of uplink transmission from vehicle n to base station m can be expressed as:

$$V_n^m = F_m h_n^m \beta_n^m. \quad (4)$$

where F_m denotes the edge spectral resource and β_n^m reflects the edge spectral ratio.

According to many existing studies [3], [33], [34], [35], [36], the result obtained after task computation typically consists of only a small number of characters. As a result, the downlink transmission time and the energy consumption can be considered negligible. The task transfer time for the offloading portion can be expressed as:

$$T_n^{tran} = \frac{\alpha_n s_n}{V_n^m}. \quad (5)$$

C. Computation Model

Local computing model: we represent the computational resources available on the vehicle as c_n^{UE} . Additionally, we assume that the vehicle's storage resources can adequately meet the local storage requirements of its tasks. Therefore, task n will be divided to determine the local computation of the subtask t_n^{loc} and the local computation time is denoted as:

$$T_{n,com}^{loc} = \frac{(1 - \alpha_n) c_n}{c_n^{ue}}. \quad (6)$$

Offloading computation model: we denote the computational resources of edge server m as C_m , the vehicle task n , after selecting the base station m that allows offloading, will perform the offloading computation for the sliced sub-tasks $task_n^{mec}$, and the base station will allocate the remaining computational resources in the server in accordance with the allocation ratio χ_n^m , so that the computation completion time of this offloading part is:

$$T_{n,com}^{mec} = \frac{\alpha_n c_n}{\chi_n^m C_m}. \quad (7)$$

D. Energy Consumption

In this paper, the system energy consumption is considered separately based on the different offloading strategies for each part of the vehicle task. Once the task partitioning is completed, the local processing of the subtasks n generates local computational energy, denoted as

$$E_{n,com}^{loc} = T_{n,com}^{loc} \cdot P_{vehicle,n}^{com}, \quad (8)$$

where $T_{n,com}^{loc}$ is the task's local computation time and $P_{vehicle,n}^{com}$ denotes the on-board chip computation power of vehicle generating task n . Furthermore, the energy consumption associated with the offloaded computation subtasks includes both the energy consumed during task uploading and the energy consumed during computation on the edge server. These energy components are denoted as:

$$E_{n,tran} = T_{n,tran} \cdot P_n^{tran}, \quad (9)$$

$$E_{n,com}^{mec} = T_{n,com}^{mec} \cdot (P_{edge,m}^{com} \cdot \chi_n^m). \quad (10)$$

Therefore, the total system energy consumption is the sum of the above local computation energy, offload transmission energy and offload computation energy:

$$E_{sum} = E_{n,com}^{loc} + E_{n,tran} + E_{n,com}^{mec}. \quad (11)$$

IV. TRANSFORMATION OF THE PROBLEM

The vehicle tasks n are divided into two parts based on the task division action α_n after generation. One part is computed locally, while the other part is offloaded to the edge server for computation via wireless transmission. As a result, the first part of the subtask completion process only involves local computation time. The second part of the subtask requires computation after transmission, which includes both transmission latency and computation latency. The time for the local computation part is represented by (6), while the total time for offloading the computation is expressed as follows:

$$T_n^{mec} = T_{n,com}^{mec} + T_n^{tran} = \frac{\alpha_n c_n}{\chi_n^m C_m} + \frac{\alpha_n s_n}{V_n^m}. \quad (12)$$

Vehicle task completion requires aggregation of the results of the two subtasks, so the start of aggregation time, i.e., the final completion time of the task, is the maximum of the two subtasks:

$$T = \max(T_{n,com}^{loc}, T_n^{mec}). \quad (13)$$

In the architecture of the MEC system, the moving vehicle transmits its real-time movement status, location, and generated

task information wirelessly to the MEC server. Then, through the monitor, based on the collected road segment information, it centrally manages the offloading and division requests of tasks under the road segment as well as the spectrum, computation, and other resources among the vehicle queues in the edge server. Efficiently performing task division and allocating limited server resources to satisfy the task offloading requests of vehicle users as much as possible is a crucial challenge in intelligent driving vehicle networks. To tackle this challenge, we propose formulating problems aiming to maximize the number of offloading tasks completed by the MEC servers, while ensuring the satisfaction of quality of service (QoS) requirements and minimizing the total system delay. Accordingly, we define the following two optimization objectives:

- a. The optimization objective is to improve the service satisfaction of all users in general. This optimization objective is obtained by averaging the difference between the maximum tolerated delay of all tasks and the final completion time of all tasks. This index essentially reflects the ability of the system to meet the user's expectation of task processing speed, and a larger difference indicates that the system performance is closer to the user's ideal expectation, and thus the user satisfaction is higher. Specifically, each task has a preset maximum tolerance delay, which is the user's mental threshold for the task completion time, and the value is related to the task attributes and task size. Users are satisfied when the actual completion time of the task is below this threshold; On the contrary, there will be dissatisfaction. Our optimization strategy aims to ensure that as many tasks as possible complete within their maximum tolerance delay and maximize the average of this difference, that is, maximize user satisfaction, through the optimal task division ratio decision and efficient edge server resource allocation.

$$P1 = \max \left(\sum_{n \in N} (d_n - \max(T_{n,com}^{loc}, T_n^{mec})) \right), \quad (14)$$

equal to:

$$P1 = \max \left(\sum_{n \in N} \left(d_n - \max \left(\frac{(1 - \alpha_n) c_n}{c_n^{ue}}, \frac{\alpha_n c_n}{\chi_n^m C_m} + \frac{\alpha_n s_n}{F_m \beta_n^m W \log_2 \left(1 + \frac{p_n^{tran} G_n^m}{\sigma^2} \right)} \right) \right) \right). \quad (15)$$

- b. Minimize the total energy consumption generated by the system while meeting the constraints of maximum task latency:

$$P2 = \min \left(\sum_{n \in N} E_{sum} \right), \quad (16)$$

equal to:

$$P2 = \min \left(\sum_{n \in N} \left(\frac{(1 - \alpha_n) c_n}{c_n^{ue}} \cdot P_{vehicle,n}^{com} + \frac{\alpha_n s_n}{V_n^m} \cdot P_n^{tran} \right) \right)$$

$$+ \frac{\alpha_n c_n}{\chi_n^m C_m} \cdot (P_{edge,m}^{com} \cdot \chi_n^m) \Big) \Big), \quad (17)$$

and satisfy the following restrictions:

$$\begin{cases} C1 : \alpha_n \in [0, 1] \\ C2 : \chi_n^m \in [0, 1]; \beta_n^m \in [0, 1] \\ C3 : \sum_{n=1}^j \chi_n^m = 1 (m \subseteq M) \\ C4 : \sum_{n=1}^j \beta_n^m = 1 (m \subseteq M) \\ C5 : P_{\min} \leq P_{vehicle,n}^{com}, P_n^{tran}, P_{edge,m}^{com} \leq P_{max} \\ C6 : T \leq d_n \end{cases} \quad (18)$$

Since both formulations of the optimization problem are non-convex due to the presence of the Heaviside step function, traditional optimization methods are not feasible without changing the original objective function [37], so we decompose P1 into two levels of subproblems. Subproblem 1 is to determine the optimal task partitioning ratio for a given user association. In the case of optimal task partitioning, subproblem 2 determines the resource allocation policy in the edge server. The problem can be solved by an iterative approach based on multi-state decomposition.

V. SOLUTIONS BASED ON DEEP REINFORCEMENT LEARNING

A. Deep Reinforcement Learning

Since the optimization problem of P1 is non-convex, traditional optimization schemes are not feasible. Moreover, the problem complexity exponentially increases with the number of vehicles in the MEC server's service area and the number of edge servers. This leads to extended solution times, which is undesirable for delay-sensitive tasks. To address this, we propose modeling the resource allocation decision-making problem as a Markov Decision Process (MDP) [35], and subsequently applying deep reinforcement learning methods to solve it [36]. The fundamental architecture of deep reinforcement learning comprises interactions between an agent and the environment [37]. The agent is implemented as a controller installed on each MEC server, while everything outside the controller is considered as the environment. Learning the optimal resource allocation policy enables efficient determination of resource allocation decisions for a given dynamic environmental state. This policy maximizes the total cumulative reward and ensures efficient resource utilization.

Based on the previous section, subproblem 1 aims to determine the optimal task division ratio for the vehicle task. This ratio ensures that the local computation latency is equivalent to the overall completion latency of the offloaded subtask.

$$T_{n,com}^{loc} = T_n^{mec}, \quad (19)$$

equal to:

$$\frac{(1 - \alpha_n) c_n}{c_n^{ue}} = \frac{\alpha_n c_n}{\chi_n^m C_m} + \frac{\alpha_n s_n}{F_m \beta_n^m W \log_2 \left(1 + \frac{p_n^{tran} G_n^m}{\sigma^2} \right)}. \quad (20)$$

The optimization objective for energy consumption is to minimize the total energy consumed across different stages. This objective eliminates the need for problem transformation, as indicated in (17). Due to the coupling between parameters

α_n and χ_n^m, β_n^m , we decompose the original problem into two sub-problems: sub-problem 1 and sub-problem 2. To solve these sub-problems, we employ a multi-state iteration algorithm based on Markov decision-making [34]. To accommodate different action strategies for each sub-problem, an intermediate state is introduced in the Markov chain. This facilitates alternating search during the Markov decision-making process.

B. Algorithmic Strategies Based on DDPG

Deep reinforcement learning algorithms can be classified into two categories: model-based and model-free. The categorization is based on whether the intelligence is capable of making predictions about the next state and reward by learning from the current environment. These algorithms can also be further categorized as stochastic or deterministic, depending on the continuity or discreteness of the actions. In vehicular networks with MEC servers, the environment exhibits dynamic changes over time due to the stochastic nature of task generation and vehicle location. This implies that the agent cannot devise an appropriate decision scheme for the subsequent state solely based on the learning strategy in the current state. However, while there are constraints on the granularity of task partitioning actions, the resource partitioning ratio can be considered as a continuous action. Moreover, the fine-grained restriction on task division actions can be accommodated through action normalization. In light of these considerations, we adopt a model-free deterministic deep reinforcement learning algorithm known as the DDPG (Deep Deterministic Policy Gradient) algorithm. This algorithm effectively tackles the challenges arising from the dynamic nature of the environment and the continuous resource partitioning in vehicular networks with MEC servers.

DDPG is an enhanced Actor-Critic algorithm that combines the strengths of both policy gradient and DQN (Deep Q-Network) algorithms, enabling the learning of policies in high-dimensional continuous action spaces. The architecture of DDPG is depicted in Fig. 1. Similar to DQN, both actors and critics employ target networks with softly updated parameters to ensure stable convergence. We employ two deep neural networks (DNNs) with identical architectures but different parameters. The evaluation network, which has real-time updated parameters, and the target network, which has parameters updated softly, facilitate the interaction between actors and critics.

The intelligent agent takes an action $\alpha(t)$ based on the current observed environment state $S(t)$. This action $\alpha(t)$ is then inputted into the environment, and receives the next environment state $S(t+1)$ and an evaluated reward $r(S(t), \alpha(t))$ given by the environment. The process of selecting an action by the intelligent agent is defined by the policy $\pi : S \rightarrow \mu(\alpha)$. The value of each state is defined as the cumulative reward obtained by discounting future rewards at each time step, denoted as $R(t)$.

$$R(t) = \sum_{i=t}^T \gamma^{i-t} \cdot r(S(i), \alpha(i)), \quad (21)$$

$$\mathcal{Q}^\mu(S(t), \alpha(t)) = \mathbb{E}_{r(i>t), S(i>t)} [R(t) | S(t), \alpha(t)]. \quad (22)$$

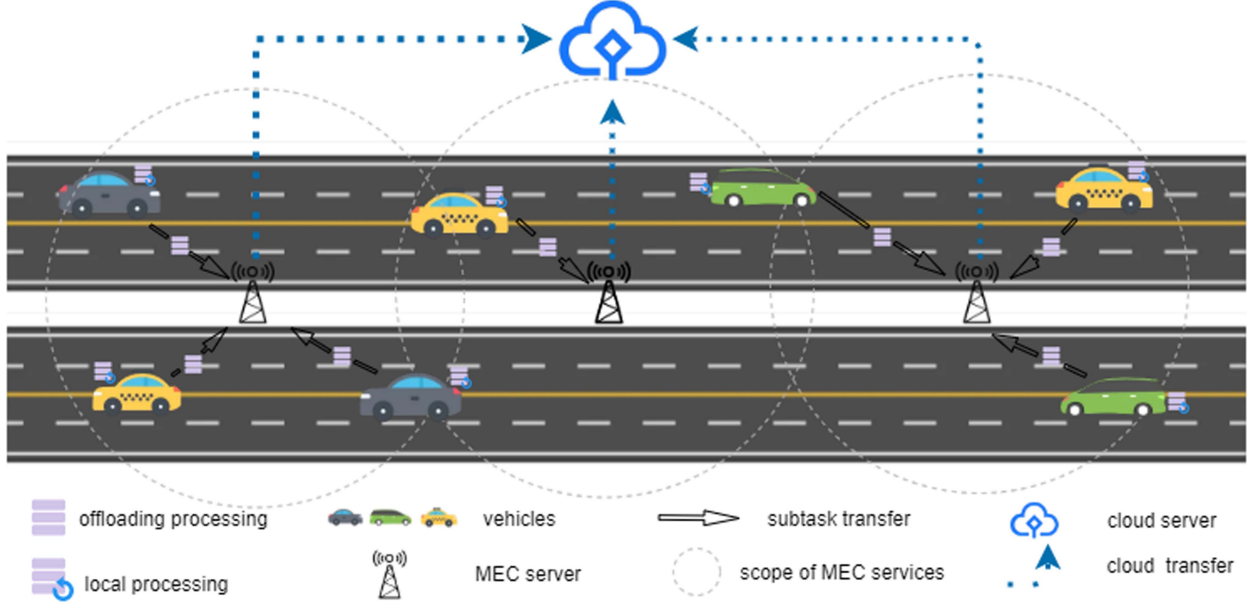


Fig. 1. Background image.

The discount factor $\gamma \in (0, 1)$ determines the extent to which future rewards are taken into account in the decision-making process. Higher value of γ implies a greater emphasis on long-term rewards, while lower value assigns more weight to immediate rewards. This discounting factor captures the notion that the impact and value of future rewards diminish as they are further in time from the current state. This relationship is expressed through the recursive formulation of Bellman's equation.

$$\begin{aligned} Q^\mu(S(t), \alpha(t)) &= \mathbb{E}_{r(S(t), \alpha(t)), S(t+1)} \\ [r(S(t), \alpha(t)) + \gamma Q^\mu(S(t+1), \mu(S(t+1)))] \end{aligned} \quad (23)$$

The current actual reward value is obtained by accumulating the reward value received from the environment with the discounted value of the evaluated reward for future states under the selected strategy.

$$y(t) = r(S(t), \alpha(t)) + \gamma Q(S(t+1), \mu(S(t+1)) | \theta^Q), \quad (24)$$

$$\mathcal{L} = \sum_{i=0}^T (y(i) - Q(S(i), \alpha(i)) | \theta^Q)^2. \quad (25)$$

The algorithm optimizes the target network by maximizing the updated action network $\mu(S) = \operatorname{argmax} Q(S, \alpha)$ through the greedy strategy on the one hand. To prevent the Q-value from diverging after updating, the strategy aims to minimize the gap $\mathcal{L}$ between the actual return value and the evaluated value. This is achieved by optimizing the target network through "soft" updating, instead of directly assigning weights to the strategy. The parameter "soft" updating method is presented as follows:

$$\theta^{\mu'} = \lambda_1 \theta^\mu + (1 - \lambda_1) \theta^{\mu'}, \quad (26)$$

$$\theta^{Q'} = \lambda_2 \theta^Q + (1 - \lambda_2) \theta^{Q'}, \quad (27)$$

where λ_1, λ_2 are the learning rates of the action network and the evaluation network, respectively, and $\lambda_1 \ll 1, \lambda_2 \ll 1$; the other $\theta^\mu(\theta^Q)$ and $\theta^{\mu'}(\theta^{Q'})$ are the network parameters of the actor(critic) corresponding to evaluation and target, respectively, which can greatly improve the stability of the algorithm's learning by allowing these target networks to update the weights of these target networks by slowly tracking the evaluating networks that have been learned.

Similar to DQN, the DDPG algorithm involved a replay buffer, which is a data space used to store the data $(S(t), \alpha(t), S(t+1), r(t))$ acquired at each iteration. The replay buffer replaces the oldest sample data with the newest sample data when it becomes full. During each time step, the actor and critic networks sample a small set of data evenly from the replay buffer to perform network updates. As DDPG is a non-policy algorithm, the replay buffer can be large, allowing the algorithm to benefit from learning across a set of uncorrelated experiences. By randomly sampling from the replay buffer, it helps break the correlation between sequential samples, thereby improving both the robustness and convergence speed of the algorithm. This approach also avoids the issue of non-independence and non-identically distributed sample populations that may arise from sequential data collection. The basic framework of the DDPG algorithm is depicted in Fig. 2.

C. Algorithmic Strategies Based on SDDPG

According to the framework structure of DDPG, we incorporate the task splitting subproblem and the resource allocation subproblem into DDPG. We formulate the multi-state iteration algorithm, SDDPG (Step iteration DDPG), based on DDPG by splitting the states and actions of the DDPG algorithm. This enables us to adapt to the iterative optimization objective for multiple actions in a partially offloading scenario and optimize

Algorithm 1: DDPG.

/*INITIALIZATION PHASE*/

Randomly initialize actor network parameter θ^μ and critic network parameter θ^Q ;Initialize target network parameter $\theta^{\mu'}$ and $\theta^{Q'}$ with weights $\theta^{\mu'} \leftarrow \theta^\mu$ and $\theta^{Q'} \leftarrow \theta^Q$;

Initialize replay buffer R.

/*INITIALIZATION PHASE*/

for each episode **do**Initialize environment variables $S(1)$;Initialize **reward**=0.**for each** t **do**Select action in the current environment based on the strategy: $\alpha(t) = \mu(S(t))$;Execute action $\alpha(t)$, get environmental reward $r(t)$ then observe new $S(t+1)$.**if** t < R **then**

Put the obtained set of experiences

 $(S(t), \alpha(t), S(t+1), r(t))$ into the replay buffer R.**else**

Replace the first saved experience with

 $(S(t), \alpha(t), S(t+1), r(t))$ Sample a random minibatch of n experiences from R;Set $y(t)$ and update critic by minimizing the $\mathcal{L}$;Update the parameter matrix in the actor and critic based on gradient values $\Delta\theta^\mu, \Delta\theta^Q$:

$$\theta^\mu \leftarrow \theta^\mu + \Delta\theta^\mu$$

$$\theta^{\mu'} = \lambda_1 \theta^\mu + (1 - \lambda_1) \theta^{\mu'}$$

$$\theta^Q \leftarrow \theta^Q + \Delta\theta^Q$$

$$\theta^{Q'} = \lambda_2 \theta^Q + (1 - \lambda_2) \theta^{Q'}$$

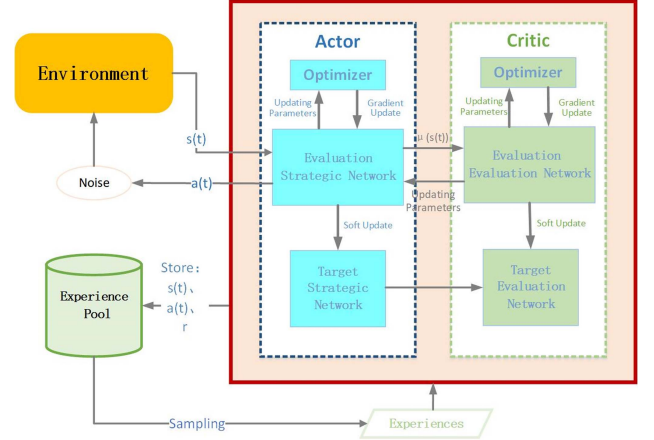
endreward = reward + $r(t)$.**end**

Fig. 2. DDPG.

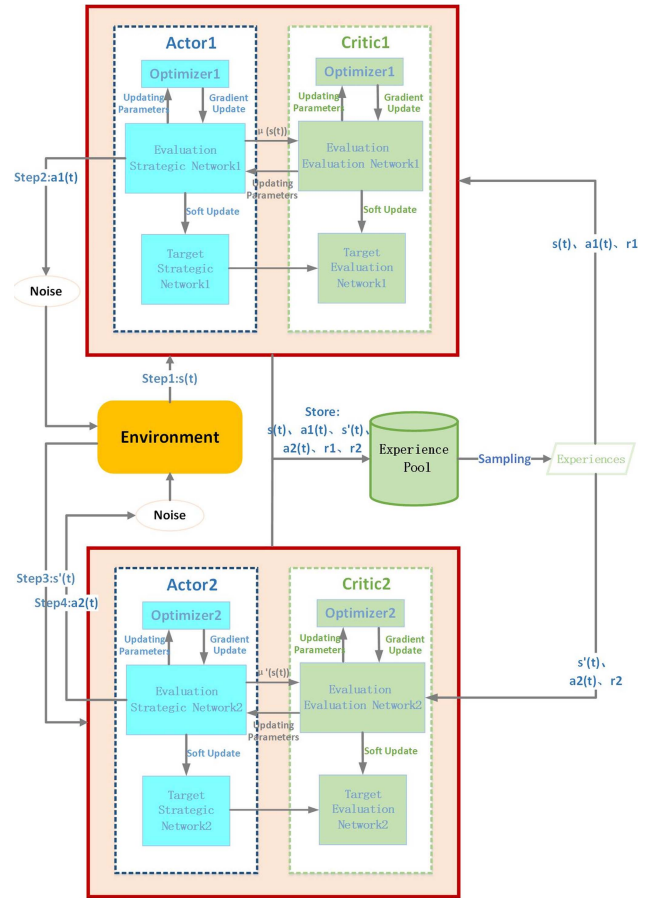


Fig. 3. SDDPG.

the two subproblems separately. The framework of SDDPG is depicted in Fig. 3.

1) *Environment and Action Splitting*: For SDDPG, we split action α into two sub-actions α_1, α_2 , and added the environment values of the intermediate states, the initial state of the environment for each cycle is shown below:

$$s(t) = \{x_1(t), x_2(t), \dots, x_j(t), c_1^s(t), c_2^s(t), \dots, c_j^s(t), s_1^s(t), s_2^s(t), \dots, s_j^s(t)\}, \quad (28)$$

where $x_j(t)$ is the coordinate position of each vehicle user in phase t , and $c_j^s(t), s_j^s(t)$ represents the amount of tasks remaining in each vehicle task in phase t and the occupied storage space, respectively. And then, the corresponding α_1 is made to this environment through α_1 's action policy network, representing the task division ratio for each vehicle task.

Then $\alpha_1(t)$ is entered into the environment variable, and the intermediate environment state $S(t)$ can be obtained according

to the task partitioning formula:

$$S(t) = \{c_1^m(t), c_2^m(t), \dots, c_j^m(t), s_1^m(t), s_2^m(t), \dots, s_j^m(t)\}. \quad (29)$$

The intermediate state represents the number of tasks and the storage space occupied by the offloaded portion of the subtasks after the task division by each vehicle user, according to α_1 . The intelligent agent takes the corresponding action α_2 towards the intermediate environment state through the action policy network of α_2 . This action represents the allocation of spectrum

and computational resources by the edge server to each vehicle task.

$$\alpha_1(t) = \{\alpha_{1,1}(t), \alpha_{1,2}(t), \dots, \alpha_{1,j}(t)\} \quad (30)$$

$$\alpha_2(t) = \{f_1(t), f_2(t), \dots, f_j(t), \chi_1(t), \chi_2(t), \dots, \chi_j(t)\}. \quad (31)$$

After providing the spectrum and computational resource allocation ratio α_2 to the environment, the computational latency and transmission latency required for completing the offloaded portion of the task are calculated. This calculation is based on the task amount and resource allocation ratio. The environment is then updated to the next stage $s(t+1)$, and the environment return reward values $r(1)$ and $r(2)$ for the action $\alpha(1)$ and action $\alpha(2)$, respectively. The experience tuple $(s(t), \alpha_1(t), S(t), \alpha_2(t), s(t+1), r_1(t), r_2(t))$ is stored in the experience replay buffer for future retrieval.

2) *Reward Value*: Based on different optimization objectives, the reward values are associated with delay time or energy consumption. Lower values of r_1 and r_2 indicate preferable strategies. For tasks sensitive to delay, r_1 and r_2 respectively represent local processing delay and offloading processing delay. The optimization objective is to minimize the expected value R of the total delay time for all vehicle tasks within the vehicular network system. Conversely, in energy-sensitive scenarios, r_1 and r_2 correspond to local energy consumption and MEC energy consumption, respectively. The optimization objective is to minimize the expected value R of the sum of vehicle and MEC energy consumption.

Latency Optimization: For α_1 , it is necessary to minimize the final completion time. The final completion time represents the latest completion time between the local computation task and the offloading computation task. Therefore, it is important to ensure that both subtasks can be completed simultaneously (without considering the backhaul time), as shown in (19). Besides, we introduce a penalty function for α_1 that accounts for the difference in completion time between the two subtasks. The reward value for this penalty function is denoted as:

$$r_t^1 = \sum_{n=1}^j |T_{n,com}^{loc} - T_n^{mec}|, \quad (32)$$

Regarding α_2 , as it does not affect the completion time of localized offloading tasks, its purpose is to unilaterally ensure that the total completion time of sub-problem 2 is minimized. This indirectly influences the allocation of action α_1 in order to reduce the overall system delay. Specifically, a lower completion time for the offloaded portion of the environment results in a lower reward. Enabling better resource allocation strategies for subtasks that offload processing. In the subsequent stage, as the completion time of the offloaded portion of the task decreases, it leads to a lower reward value r_t^1 . Consequently, the value of α_1 is indirectly increased, resulting in an increase in the amount of offloaded tasks and a decrease in the amount of localized offloading tasks. This optimization of the system's offloading and resource allocation scheme can be expressed by the reward

value of α_2 as follows:

$$r_t^2 = T_n^{mec}. \quad (33)$$

The overall performance of the algorithm is evaluated based on the return value R_t . The value of R_t represents our optimization objective P1. By calculating the overall average of the time that each task can be completed in advance, this average value serves as our overall reward, which reflects the extent to which all tasks are completed ahead of schedule. We express this as user service satisfaction and utilize the SDDPG algorithm to maximize the average user satisfaction for all tasks.

Energy consumption optimization: Energy consumption is a crucial factor in mobile edge computing, which not only affects the battery life of the device, but also directly relates to the overall operating cost of the system. Specifically, our algorithm takes into account both local and offloading energy consumption during the decision-making process. The local energy consumption mainly refers to the energy consumed by the vehicle when executing the computing task locally, while the offloading energy consumption refers to the additional energy consumption generated by offloading the task to the edge server for processing, which usually includes the energy consumption during data transmission and the energy consumption when the edge server processes the task. In order to optimize energy consumption, our algorithm incorporates energy consumption considerations into the task division ratio decision and edge server resource allocation. In the task partitioning ratio decision stage, the algorithm will evaluate the impact of different partitioning ratios on local energy consumption and offloading energy consumption, and try to find a balance point to minimize the overall energy consumption. In the resource allocation stage of edge servers, the algorithm will allocate resources efficiently according to the current network conditions and the energy consumption model of edge servers to further reduce the offloading energy consumption.

To facilitate the widespread adoption of mobile edge computing, algorithms must demonstrate robust performance under conditions of energy supply scarcity. By modifying the reward function of the algorithm, we direct its optimization towards reducing energy consumption. When the reward is set as energy consumption, the algorithm meticulously analyzes the energy consumption patterns of various tasks and dynamically adjusts task execution strategies based on these patterns. Rather than focusing solely on the energy consumption of individual tasks, it adopts a holistic system perspective, optimizing task scheduling and computational resource allocation to minimize overall energy consumption. This holistic optimization strategy enables the SDDPG algorithm to exhibit enhanced efficiency in environments with limited energy supplies. Specifically, we decompose the energy consumption reward into two components: r_e^1 , representing the transmission energy consumption of task allocation actions, and r_e^2 , representing the computational energy consumption of resource allocation actions:

$$r_e^1 = E_{n,tran}, \quad (34)$$

$$r_e^2 = E_{n,com}^{mec}, \quad (35)$$

In detail, the SDDPG algorithm's characteristics in energy consumption optimization are primarily manifested in the following aspects: First, it leverages deep learning techniques to capture the complex relationships among tasks, thereby enabling more accurate predictions of energy consumption during the execution of different tasks. Second, through continuous iteration and strategy updates, the algorithm progressively converges towards the optimal solution, ensuring minimal energy consumption within limited computational resources. Lastly, the algorithm possesses excellent scalability and adaptability, allowing for flexible adjustments according to different application scenarios and requirements. The overall reward value R_e , which is the sum of r_e^1 and r_e^2 , represents the total energy consumption of the edge server.

$$R_e = r_e^1 + r_e^2. \quad (36)$$

In conclusion, we anticipate that the SDDPG algorithm will exhibit significant capabilities and advantages in energy consumption optimization when addressing energy supply shortages. By precisely analyzing task energy consumption patterns, employing holistic optimization strategies, and leveraging deep learning techniques, the algorithm provides robust technical support and assurance for the widespread adoption of mobile edge computing. The SDDPG algorithm exhibits notable advantages in delay and energy consumption optimization through the application of distinct reward values to provide independent feedback for different actions. In terms of delay optimization, the independent feedback mechanism of the SDDPG algorithm simplifies the training process. By assigning explicit reward values to various actions, the algorithm can more directly evaluate the impact of each action on delay, thereby effectively adjusting strategies to reduce the time required for task completion. Furthermore, this mechanism mitigates the interference of cumulative errors in the training process, ensuring that the algorithm accurately approximates the optimal solution during iterations. Consequently, the SDDPG algorithm demonstrates a more convergent characteristic in delay optimization.

3) *Algorithmic Adaptation*: In the SDDPG algorithm, the calculation methods for task latency and system energy consumption are independent. To address different optimization objectives in various environments, we incorporate a parameter $\varsigma \in [0, 1]$ to adjust the system's sensitivity accordingly. For instance, in high-speed road scenarios where prompt decision-making is crucial and latency is more critical, we can increase the parameter ς to optimize the algorithm towards minimizing latency. Conversely, in remote mountainous areas where low energy consumption can extend server usage time, the parameter ς can be decreased to optimize the algorithm with a specific focus on reducing energy consumption. This optimization enables a more versatile application of the algorithm model.

$$r_1 = \varsigma r_t^1 + (1 - \varsigma) r_e^1, \quad (37)$$

$$r_2 = \varsigma r_t^2 + (1 - \varsigma) r_e^2, \quad (38)$$

$$R = r_1 + r_2. \quad (39)$$

4) *Network Model*: SDDPG splits a single action into two sequentially related actions to finely control task partitioning and

Algorithm 2: SDDPG.

```

/*INITIALIZATION PHASE*/
Randomly initialize actor network parameter  $\theta_1^\mu, \theta_2^\mu$  and
critic network parameter  $\theta_1^Q, \theta_2^Q$ ;
Initialize target network parameter  $\theta_1^{\mu'}, \theta_2^{\mu'}$  and  $\theta_1^{Q'}, \theta_2^{Q'}$ 
with weights  $\theta_1^{\mu'} \leftarrow \theta_1^\mu, \theta_2^{\mu'} \leftarrow \theta_2^\mu, \theta_1^{Q'} \leftarrow \theta_1^Q$  and
 $\theta_2^{Q'} \leftarrow \theta_2^Q$ ;
Initialize replay buffer R.
/*INITIALIZATION PHASE*/
for each episode do
  Initialize environment variables  $s(1)$ ;
  Initialize reward1=0;
  Initialize reward2=0;
  for each t do
    Select action in the current environment based on
    the actor1 evaluation strategy:  $\alpha_1(t) = \mu_1(s(t))$ ;
    Execute action  $\alpha_1(t)$ , and observe new state  $S(t)$ ;
    Select action in the current environment based on
    the actor2 evaluation strategy:  $\alpha_2(t) = \mu_2(s(t))$ ;
    Execute action  $\alpha_2(t)$ , observe new state
     $s(t+1)$ , and get environmental reward  $r_1(t)$ ,
     $r_2(t)$  Based on  $\alpha_1$  and  $\alpha_2$ .
    if t < R then
      Put the obtained set of experiences
       $(s(t), \alpha_1(t), S(t), \alpha_2(t), s(t+1), r_1(t),$ 
       $r_2(t))$  into the replay buffer R.
    else
      Replace the first saved experience with
       $(s(t), \alpha_1(t), S(t), \alpha_2(t), s(t+1), r_1(t),$ 
       $r_2(t))$ ;
      Sample a random minibatch of n experiences
      from R;
      Set  $y_1(t)$  and  $y_2(t)$  update critic by minimizing
      the loss:  $\mathcal{L}_1, \mathcal{L}_2$ ;
      Update the parameter matrix in the actor and
      critic based on gradient values  $\Delta\theta_1^\mu, \Delta\theta_1^{Q'}$  and
       $\Delta\theta_2^\mu, \Delta\theta_2^{Q'}$ :
       $\theta_1^\mu \leftarrow \theta_1^\mu + \Delta\theta_1^\mu$ 
       $\theta_1^{\mu'} = \lambda_1 \theta_1^\mu + (1 - \lambda_1) \theta_1^{\mu'}$ 
       $\theta_1^Q \leftarrow \theta_1^Q + \Delta\theta_1^Q$ 
       $\theta_1^{Q'} = \lambda_2 \theta_1^Q + (1 - \lambda_2) \theta_1^{Q'}$ 
       $\theta_2^\mu \leftarrow \theta_2^\mu + \Delta\theta_2^\mu$ 
       $\theta_2^{\mu'} = \lambda_3 \theta_2^\mu + (1 - \lambda_3) \theta_2^{\mu'}$ 
       $\theta_2^Q \leftarrow \theta_2^Q + \Delta\theta_2^Q$ 
       $\theta_2^{Q'} = \lambda_4 \theta_2^Q + (1 - \lambda_4) \theta_2^{Q'}$ 
    end
    reward1 = reward1 +  $r_1(t)$ 
    reward2 = reward2 +  $r_2(t)$ 
  end

```

resource allocation during task offloading. This enhancement not only increases the complexity of the algorithm but also significantly improves its performance in terms of delay and energy consumption optimization. Specifically, our algorithm comprises eight neural networks, corresponding to the policy and evaluation networks for two actions (task partitioning ratio

a_1 and resource allocation a_2), as well as their corresponding target networks. The close interplay and collaboration among these networks are crucial for achieving algorithm optimization. Initially, the Actor current network for a_1 is responsible for generating the task partitioning ratio based on the current state. This ratio determines the distribution of tasks between local computation and offloading to the edge server. Subsequently, this action is fed back to the environment, which generates an intermediate state based on this ratio. Based on this intermediate state, the Actor current network for a_2 generates the resource allocation strategy a_2 , which determines the amount of resources required for the tasks transmitted to the edge server. Then, a_2 is fed back to the environment to execute the task offloading, resulting in the next state and reward.

The evaluation networks (critic networks) play a vital role in the algorithm. They estimate the expected reward for taking a specific action in the current state. For a_1 and a_2 , we have designed separate critic current networks and critic target networks. These networks receive the current state, action, and the next state generated by the environment as inputs and output the Q-value (i.e., the expected reward) for that action. Notably, for delay optimization, the reward can be the reduction in task completion time; for energy consumption optimization, the reward can be the decrease in energy consumption, considering factors such as processing and transmission energy. These rewards are used to train the critic networks to update the Q-value estimates and guide the actor networks to generate better action policies. During the training process, to maintain the stability and efficiency of the algorithm, we adopt the "soft" update strategy from DDPG to periodically update the parameters of the target networks. Specifically, the parameters of the target networks are updated using a learning rate parameter. This update method ensures that the target networks do not change too rapidly during training, thereby contributing to the stable convergence of the algorithm. For delay and energy consumption optimization, we achieve this by adjusting the calculation of rewards. In delay optimization, the reward directly reflects the reduction in task completion time; in energy consumption optimization, the reward considers factors such as processing and transmission energy consumption. When these rewards are used to train the critic networks, the parameters of the Actor networks can be updated through backpropagation, causing the parameters of the action networks to gradually approach the optimal strategy in accordance with our expected goals.

5) *Complexity Analysis*: The time complexity of the SIDDPG algorithm primarily hinges upon several factors: the dimensions of two state spaces, the dimensions of two action spaces, the complexity of the corresponding network architectures, and the scale of training data. Specifically, the dimensions of the two state spaces, as shown in (28) and (29), are $3n$ and $2n$, respectively; whereas the dimensions of the two action spaces, as depicted in (30) and (31), are n and $2n$, respectively. The network architecture and the scale of training data, as integral components of the algorithm's complexity, further encompass the following three aspects:

Network Forward Propagation: During each iteration, forward propagation is conducted on the current state to compute

the policy and value functions. The complexity of this part is $O(Ltn)$, where L_1 and L_2 represent the number of layers in the evaluator and actor networks, respectively, t denotes the number of neurons per layer, and n signifies the dimension of the state space.

Policy Update: The parameters of the policy network are updated based on the gradients of the value function. The complexity of this section depends on the choice of optimization algorithm but generally scales proportionally to the network architecture complexity, i.e., $O(Ltm)$, where m is the dimension of the action space.

Value Function Update: The value function is updated using experience replay and a target network. The complexity of this part primarily hinges on the scale of training data and the complexity of the network architecture, denoted as $O(LtS)$. Notably, S represents the number of samples corresponding to the scale of training data.

Synthesizing the above analyses, the SIDDPG algorithm traverses two action states and two environmental states in each iteration, rendering its complexity approximately representable as: $O(L_1t(N + M) + L_2t(N + M) + L_1tS + L_2tS) = O((L_1 + L_2)t(3n + 2n + n + 2n) + (L_1 + L_2)tS)$, which simplifies to $O((L_1 + L_2)t(7n + S))$. In the primary experiments discussed later, based on parameter comparisons, the number of vehicles n is selected as 50, the scale of training data S is chosen as 16 samples per iteration. Additionally, the evaluator and actor networks for the two actions have 4 and 3 layers, respectively (the number of layers and neurons per layer differ, and specific values are not included here for simplification).

It is worth mentioning that there is also a certain relationship between the algorithm's complexity and the number of edge servers, albeit indirect. This relationship manifests through its influence on the algorithm's runtime environment and resource allocation.

VI. SIMULATION RESULTS

In this section, we present the experimental results of the SIDDPG algorithm in various MEC scenarios to validate the benefits of the offloading and resource allocation schemes proposed by this algorithm. In this study, we specifically showcase the convergence of the SIDDPG algorithm and evaluate its performance in terms of multidimensional resource management, energy consumption, and other relevant metrics. These assessments are conducted to facilitate comparative analysis and provide insights into the algorithm's effectiveness.

We aim to replicate the real-world environment of a two-way highway. We consider vehicles accessing the highway at a speed range of 20-30 m/s and offloading tasks to edge servers located along a 5 km stretch of the highway. Road test units are evenly distributed along the middle of the road, and there is some overlap in the service areas of neighboring road test units to ensure a seamless connection for vehicular services. Vehicles within their respective service ranges offload tasks to edge servers. If a vehicle travels from one service range to another, it offloads tasks to the next edge server based on the direction of travel. All

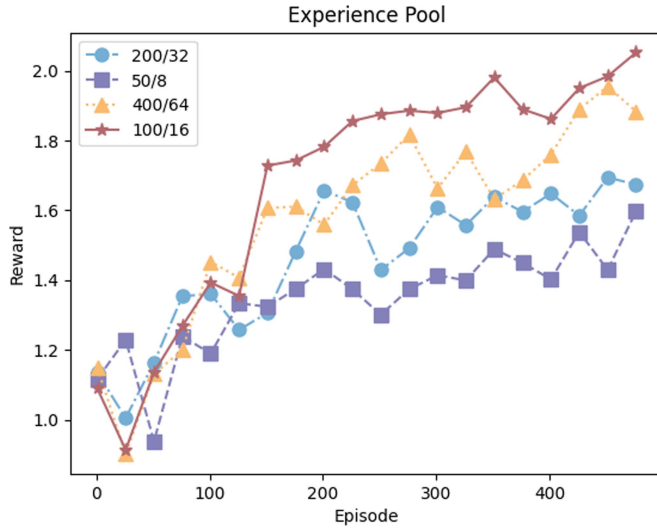


Fig. 4. Experience Pool.

edge servers have equal computing power and total bandwidth, while there may be slight variations in the computing power of on-board services. The transmission resource from the vehicle to the edge server is defined as $B \cdot \log_2(1 + \frac{p_n^{tran} \cdot \tau^2}{N})$, where the bandwidth B is 1MHz, the upload power p of the vehicle is $45w$, τ is the distance-dependent channel gain between the vehicle and the server, and the white noise N value of the communication is $1 \times 10(-11)dB$, the Vehicles randomly generate tasks about $2M$ in size to offload the access with the edge server. The maximum tolerable delay for a task is randomly 2.5-3.5s, with a positive correlation to the task size.

We conducted an evaluation of four different ratios between the capacity of the experience pool and the number of samples. After analyzing the results shown in Fig. 4, we determined that the capacity value of 100/16 best suited our experimental requirements. Therefore, we set the capacity of the experience pool to 100, and each sampling process involved 16 samples. In addition, we considered the optimization process of the algorithm under varying levels of environmental complexity. As shown in Fig. 5, excessively low complexity resulted in significant fluctuations, while excessively high complexity impeded the convergence speed of the algorithm. However, within the context of a system consisting of 5 servers and 50 vehicles, we successfully achieved an optimal balance between reward value and convergence speed. Using the aforementioned parameters, we will evaluate the algorithm's performance in terms of service satisfaction, task completion rate, and energy consumption.

We initiate the evaluation by comparing the efficiency of the partial offloading scheme with the traditional overall offloading scheme. Fig. 6 illustrates the average service satisfaction of all vehicles under different offloading methods. Service satisfaction is determined by calculating the difference between the actual completion time of a vehicle's task and the maximum permissible delay for that task within the system. Positive values indicate that the task is completed within the allowed time frame, with higher values indicating earlier completion.

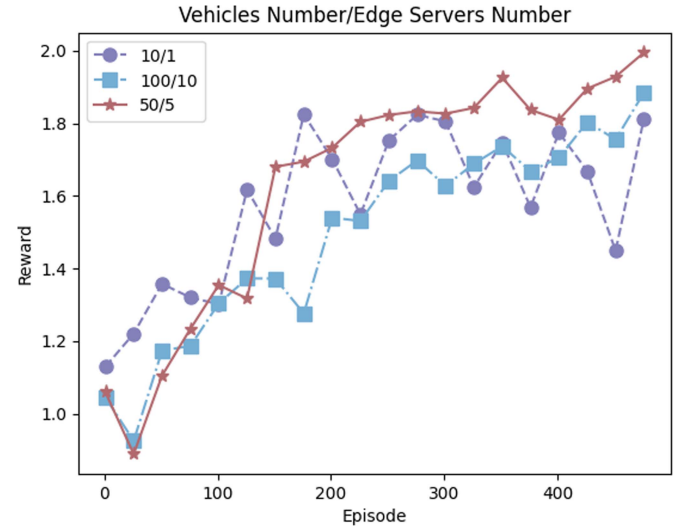


Fig. 5. Model Complexity.

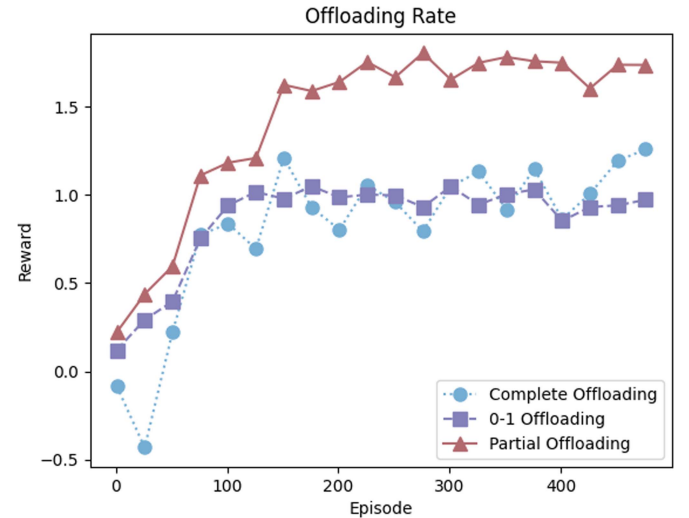


Fig. 6. Offloading Rate.

Conversely, negative values indicate that the task exceeds the maximum permissible delay. The partial offloading scheme exhibits significant improvements in task delay by utilizing the computational resources of the vehicles themselves for auxiliary processing. However, the involvement of the local vehicle's computational power in the processing increases the complexity of the algorithm. As a result, the convergence of the algorithm occurs approximately 50-100 rounds later compared to the full offloading and complete offloading schemes.

The MADDPG and HDDPG algorithms are optimization algorithms derived from the DDPG algorithm. While they demonstrate excellent efficiency under the overall offloading scheme, they do not fully consider the performance advantages in specific offloading scenarios. To address this, we compare these algorithms in terms of service satisfaction, task completion rate, and energy consumption. The corresponding results are presented in Figs. 7, 8, and 9. The task completion rate indicates the percentage of tasks within the system that can be completed

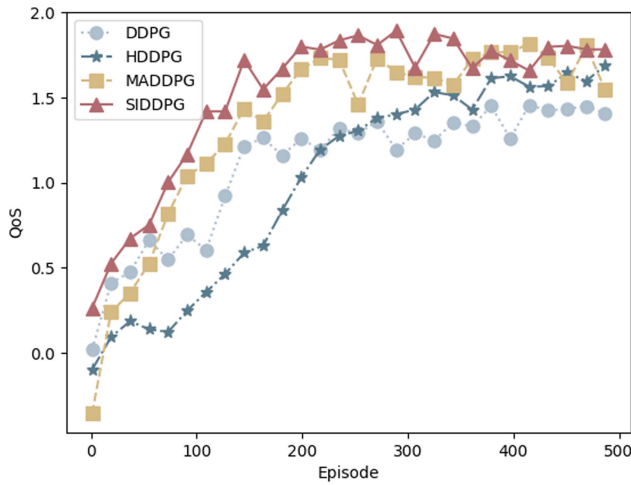


Fig. 7. Customer satisfaction.

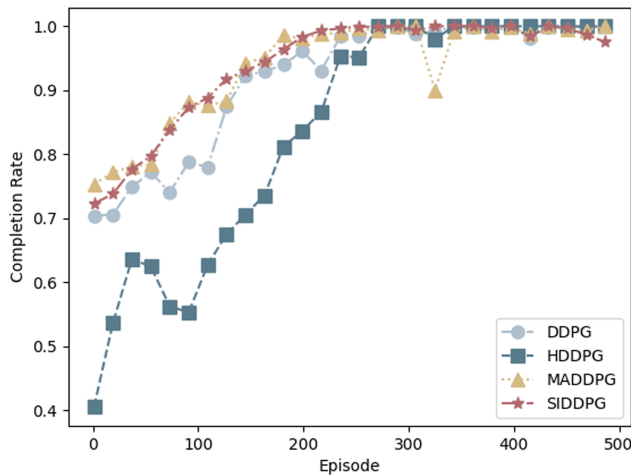


Fig. 8. Completion rate.

within the maximum permissible time. Energy consumption takes into account the overall energy consumption of the system, which includes the computational energy consumption of the vehicles, the transmission energy consumption of the tasks, and the computational energy consumption of the edge servers.

As depicted in the Fig. 7, the SDDPG algorithm generally outperforms the other three algorithms in terms of user satisfaction. It also converges faster than the DDPG and HDDPG algorithms, exhibiting more than a 10% improvement in user satisfaction after convergence.

In our scenario, the tasks generated by vehicles have different priorities, with some tasks requiring immediate processing, such as emergency situation discrimination and traffic signal recognition. The ability of intelligent vehicles to make timely judgments becomes crucial in such cases. To assess this ability, we introduce the indicator of task on-time completion rate. As depicted in Fig. 8, both SDDPG and MADDPG algorithms demonstrate a high task completion rate. However, after convergence, SDDPG exhibits a more stable approach towards achieving a high completion rate for all tasks.

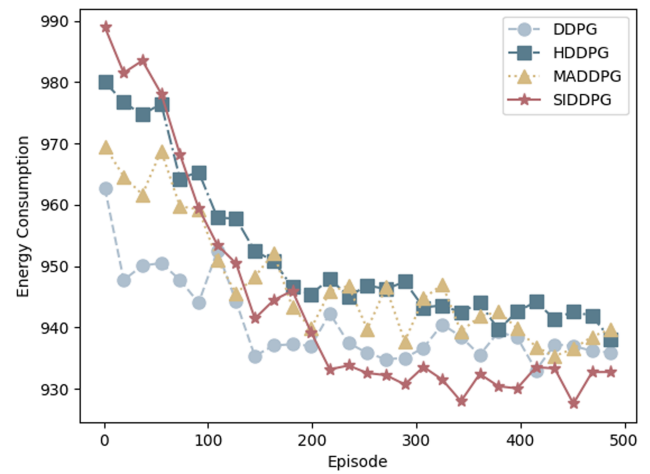


Fig. 9. Power consumption.

When considering energy consumption, the SDDPG algorithm initially shows higher energy consumption compared to other algorithms due to the involvement of the vehicle in offloading tasks. However, SDDPG efficiently balances the processing time and reduces transmission overhead during the subsequent convergence process, as depicted in Fig. 9. As a result, the energy consumption of the SDDPG algorithm is ultimately reduced by more than 2% compared to the DDPG, MADDPG, and HDDPG algorithms.

VII. CONCLUDE

This paper proposes the SDDPG algorithm, an enhancement of the DDPG algorithm, designed to address efficiency partial offloading and resource management issues in MEC-enabled IoT networks. The SDDPG algorithm offers faster convergence and more effective decision-making for multi-user collaborative offloading management on complex roads. It also optimizes performance for high-speed vehicles, particularly in emergency scenarios, by improving response times and reducing overall system energy consumption.

Future research will focus on further reducing vehicle energy consumption and resolving load-balancing challenges in multi-tasking environments.

REFERENCES

- [1] V. D. Tuong, T. P. Truong, T.-V. Nguyen, W. Noh, and S. Cho, "Partial computation offloading in NOMA-assisted mobile-edge computing systems using deep reinforcement learning," *IEEE Internet Things J.*, vol. 8, no. 17, pp. 13 196–13 208, Sep. 2021.
- [2] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A survey on enabling technologies, protocols, and applications," *IEEE Commun. Surv. Tut.*, vol. 17, no. 4, pp. 2347–2376, Fourth Quarter 2015.
- [3] X. Chen, L. Jiao, W. Li, and X. Fu, "Efficient multi-user computation offloading for mobile-edge cloud computing," *IEEE/ACM Trans. Netw.*, vol. 24, no. 5, pp. 2795–2808, Oct. 2016.
- [4] H. Xue, B. Huang, M. Qin, H. Zhou, and H. Yang, "Edge computing for Internet of Things: A survey," in *Proc. Int. Conf. Internet Things-IEEE Green Comput. Commun.-IEEE Cyber Phys. Social Comput.-IEEE Smart Data-IEEE Congr. Cybermatics*, 2020, pp. 755–760.

- [5] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surv. Tut.*, vol. 19, no. 4, pp. 2322–2358, Fourth Quarter 2017.
- [6] C. Ren, G. Zhang, X. Gu, and Y. Li, "Computing offloading in vehicular edge computing networks: Full or partial offloading," in *Proc. IEEE 6th Inf. Technol. Mechatron. Eng. Conf.*, 2022, pp. 693–698.
- [7] P. Mach and Z. Becvar, "Mobile edge computing: A survey on architecture and computation offloading," *IEEE Commun. Surv. Tut.*, vol. 19, no. 3, pp. 1628–1656, Third Quarter 2017.
- [8] M. Feng, M. Krunz, and W. Zhang, "Joint task partitioning and user association for latency minimization in mobile edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 70, no. 8, pp. 8108–8121, Aug. 2021.
- [9] Z. Li, Z. Zheng, S. Guo, B. Guo, F. Xiao, and K. Ren, "Disguised as privacy: Data poisoning attacks against differentially private crowdsensing systems," *IEEE Trans. Mobile Comput.*, vol. 22, no. 9, pp. 5155–5169, Sep. 2023.
- [10] X. Xu, H. Li, W. Xu, Z. Liu, L. Yao, and F. Dai, "Artificial intelligence for edge service optimization in Internet of Vehicles: A survey," *Tsinghua Sci. Technol.*, vol. 27, no. 2, pp. 270–287, 2022.
- [11] X. Wang, Y. Han, V. C. M. Leung, D. Niyato, X. Yan, and X. Chen, "Convergence of edge computing and deep learning: A comprehensive survey," *IEEE Commun. Surv. Tut.*, vol. 22, no. 2, pp. 869–904, Second Quarter 2020.
- [12] G. Qiao, S. Leng, K. Zhang, and Y. He, "Collaborative task offloading in vehicular edge multi-access networks," *IEEE Commun. Mag.*, vol. 56, no. 8, pp. 48–54, Aug. 2018.
- [13] Y. M. Saputra, D. T. Hoang, D. N. Nguyen, E. Dutkiewicz, D. Niyato, and D. I. Kim, "Distributed deep learning at the edge: A novel proactive and cooperative caching framework for mobile edge networks," *IEEE Wireless Commun. Lett.*, vol. 8, no. 4, pp. 1220–1223, Aug. 2019.
- [14] K. Zhang, Y. Mao, S. Leng, Y. He, and Y. Zhang, "Mobile-edge computing for vehicular networks: A promising network paradigm with predictive off-loading," *IEEE Veh. Technol. Mag.*, vol. 12, no. 2, pp. 36–44, Jun. 2017.
- [15] H. Peng and X. Shen, "Multi-agent reinforcement learning based resource management in MEC- and UAV-assisted vehicular networks," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 1, pp. 131–141, Jan. 2021.
- [16] F. Busacca, G. Faraci, C. Grasso, S. Palazzo, and G. Schembra, "Designing a multi-layer edge-computing platform for energy-efficient and delay-aware offloading in vehicular networks," *Comput. Netw.*, vol. 198, 2021, Art. no. 108330. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1389128621003315>
- [17] G. Mitsis, E. E. Tsiropoulou, and S. Papavassiliou, "Price and risk awareness for data offloading decision-making in edge computing systems," *IEEE Syst. J.*, vol. 16, no. 4, pp. 6546–6557, Dec. 2022.
- [18] P. Charatsaris, M. Salcido, M. Diamanti, A. M. Ali, E. E. Tsiropoulou, and S. Papavassiliou, "Agora: A multi-provider edge computing resource management and pricing framework," in *Proc. Int. Wirel. Commun. Mobile Comput.*, 2024, pp. 1685–1690.
- [19] M. Diamanti, C. Pelekis, E. E. Tsiropoulou, and S. Papavassiliou, "Delay minimization for rate-splitting multiple access-based multi-server MEC offloading," *IEEE/ACM Trans. Netw.*, vol. 32, no. 2, pp. 1035–1047, Apr. 2024.
- [20] E. Cuervo et al., "Maui: Making smartphones last longer with code offload," in *Proc. 8th Int. Conf. Mobile Syst. Appl. Serv.*, New York, NY, USA, 2010, pp. 49–62, doi: [10.1145/1814433.1814441](https://doi.org/10.1145/1814433.1814441).
- [21] Q. Wu, H. Liu, R. Wang, P. Fan, Q. Fan, and Z. Li, "Delay-sensitive task offloading in the 802.11p-based vehicular fog computing systems," *IEEE Internet Things J.*, vol. 7, no. 1, pp. 773–785, 2020.
- [22] Y. Dai, D. Xu, S. Maharjan, and Y. Zhang, "Joint load balancing and offloading in vehicular edge computing and networks," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4377–4387, Jun. 2019.
- [23] Z. Kuang, L. Li, J. Gao, L. Zhao, and A. Liu, "Partial offloading scheduling and power allocation for mobile edge computing systems," *IEEE Internet Things J.*, vol. 6, no. 4, pp. 6774–6785, Aug. 2019.
- [24] T. Yi, G. Zhang, K. Wang, and K. Yang, "Joint program partitioning and resource allocation for completion time minimization in multi-MEC systems," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 3, pp. 1932–1948, May/Jun. 2022.
- [25] X. Deng, J. Yin, P. Guan, N. N. Xiong, L. Zhang, and S. Mumtaz, "Intelligent delay-aware partial computing task offloading for multiuser Industrial Internet of Things through edge computing," *IEEE Internet Things J.*, vol. 10, no. 4, pp. 2954–2966, Feb. 2023.
- [26] Y. Wang, M. Sheng, X. Wang, L. Wang, and J. Li, "Mobile-edge computing: Partial computation offloading using dynamic voltage scaling," *IEEE Trans. Commun.*, vol. 64, no. 10, pp. 4268–4282, Oct. 2016.
- [27] C. You, K. Huang, H. Chae, and B.-H. Kim, "Energy-efficient resource allocation for mobile-edge computation offloading," *IEEE Trans. Wireless Commun.*, vol. 16, no. 3, pp. 1397–1411, Mar. 2017.
- [28] S. E. Mahmoodi, R. N. Uma, and K. P. Subbalakshmi, "Optimal joint scheduling and cloud offloading for mobile applications," *IEEE Trans. Cloud Comput.*, vol. 7, no. 2, pp. 301–313, Second Quarter 2019.
- [29] Z. Ning, P. Dong, X. Kong, and F. Xia, "A cooperative partial computation offloading scheme for mobile edge computing enabled Internet of Things," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4804–4814, Jun. 2019.
- [30] M. Feng, M. Krunz, and W. Zhang, "Task partitioning and user association for latency minimization in mobile edge computing networks," in *Proc. IEEE Conf. Comput. Commun. Workshops*, 2021, pp. 1–6.
- [31] M. Qin et al., "Service-oriented energy-latency tradeoff for IoT task partial offloading in MEC-enhanced multi-RAT networks," *IEEE Internet Things J.*, vol. 8, no. 3, pp. 1896–1907, Feb. 2021.
- [32] C. Chen et al., "Delay-optimized V2V-based computation offloading in urban vehicular edge computing and networks," *IEEE Access*, vol. 8, pp. 18 863–18 873, 2020.
- [33] L. Liu, Z. Chang, X. Guo, S. Mao, and T. Ristaniemi, "Multiobjective optimization for computation offloading in fog computing," *IEEE Internet Things J.*, vol. 5, no. 1, pp. 283–294, Feb. 2018.
- [34] Y. Mao, J. Zhang, and K. B. Letaief, "Joint task offloading scheduling and transmit power allocation for mobile-edge computing systems," in *Proc. IEEE Wirel. Commun. Netw. Conf.*, 2017, pp. 1–6.
- [35] J. Zhang et al., "Joint resource allocation for latency-sensitive services over mobile edge computing networks with caching," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4283–4294, Jun. 2019.
- [36] C. Wang, C. Liang, F. R. Yu, Q. Chen, and L. Tang, "Computation offloading and resource allocation in wireless cellular networks with mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 16, no. 8, pp. 4924–4938, Aug. 2017.
- [37] H. Peng and X. Shen, "Deep reinforcement learning based resource management for multi-access edge computing in vehicular networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 4, pp. 2416–2428, Fourth Quarter 2020.



Shujuan Tian received the PhD degree from Central South University, in 2016. From 2019 to 2020, she was a postdoctoral researcher with the Computer Science department of Illinois Institute of Technology (IIT), and cooperated with Prof. Pengjun Wan. She is currently a professor with the School of Computer Science & School of Cyberspace Security, Xiangtan University. Her research interests include Mobile Edge Computing, VANET, Internet of Things, Wireless Signal Control and Processing, etc.interests.



Xinjie Zhu is currently working toward the MS degree with the School of Computer Science & School of Cyberspace Security, Xiangtan University, Xiangtan, China. His current research interests include task unloading in moving edge computing.



Bochao Feng received the MS degree from the Guilin University of Technology, Guangxi, China. He is currently working toward the PhD degree with Xiangtan University, Xiangtan, China. His research interests include mobile edge computing and resource pricing approaches.

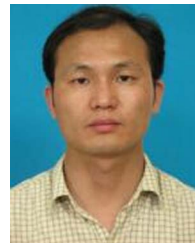


Haolin Liu (Member, IEEE) received the BEng and PhD degrees from Sichuan University, Chengdu, China, in 2010 and 2015, respectively. He is currently an associate professor with the Key Laboratory of Hunan Province for Internet of Things and Information Security and School of Computer Science, Xiangtan University, Xiangtan, China. His research interests include mobile edge computing, wireless sensor networks, and the Internet of Things.



Zhirun Zheng received the BS degree from the School of Mathematics and Statistics, Henan University of Science and Technology, China, in 2017, and the PhD degree from the School of Mathematics and Computational Science, Xiangtan University, China, in 2024. From 2022 to 2023, he visited the BroadBand Communications Research (BBCR) Lab, Department of Electrical and Computer Engineering, University of Waterloo, Canada. Since October 2024, he is a postdoctoral fellow with the Department of Artificial Intelligence, Ajou University, South Korea.

His research interests are in the areas of security and privacy in Crowdsensing systems, artificial intelligence, and mobile networks. He serves as TPC members of many conferences, including GLOBECOM and WCSP.



Zhetao Li (Member, IEEE) received the BEng degree in electrical information engineering from Xiangtan University, in 2002, the MEng degree in pattern recognition and intelligent systems from Beihang University in 2005, and the PhD degree in computer application technology from Hunan University in 2010. He is a professor with the College of Computers, Jinan University. From 2013 to 2014, he was a postdoc in wireless networks at Stony Brook University. He is a member of CCF.